

Performance Analysis of Sufficient-Statistics OLS

sufficient-regression design note

June 30, 2026

1 Cost model

Let p be the number of columns in the design matrix, including any intercept column. Let n be the number of existing rows, m the number of appended rows, w a rolling-window size, and T the number of rolling windows evaluated.

This note uses dense linear algebra costs:

$$\text{one row contribution } xx^T = \Theta(p^2), \quad \text{one dense solve or factorization} = \Theta(p^3).$$

The $\Theta(p^2)$ row cost includes the dominant outer product needed for $G = X^T X$; the $h = X^T y$ contribution costs only $\Theta(p)$. Recomputing sufficient statistics from r dense rows therefore costs

$$C_{stats}(r, p) = \Theta(rp^2).$$

Computing coefficients after the statistics are available costs $\Theta(p^3)$ with a standard dense solver. Thus a complete batch OLS fit from raw rows costs

$$C_{fit}(r, p) = \Theta(rp^2 + p^3).$$

The analysis separates statistic maintenance from coefficient solving. That separation is intentional: sufficient statistics remove repeated scans over rows, but a dense coefficient solve is still present whenever coefficients are requested.

2 Append case: one million existing rows plus 1000 new rows

Consider an already summarized data set with

$$n = 1,000,000, \quad m = 1,000.$$

The task is to incorporate m new rows into the existing OLS state.

2.1 Final coefficients requested once

If an implementation discards the old summary and recomputes from all raw rows after the append, the cost is

$$C_{raw,final} = \Theta((n + m)p^2 + p^3).$$

If an implementation keeps sufficient statistics, it only forms the contributions of the m appended rows and then solves:

$$C_{suff,final} = \Theta(mp^2 + p^3).$$

Theorem 1 (Append speedup for one final solve). *Ignoring the common solve term, sufficient-statistics append gives a statistics-maintenance speedup of*

$$\Theta\left(\frac{(n+m)p^2}{mp^2}\right) = \Theta\left(\frac{n+m}{m}\right).$$

For $n = 1,000,000$ and $m = 1,000$, this ratio is

$$\frac{1,001,000}{1,000} = 1001.$$

Proof. Both methods need the same final dense solve after their statistics are available. The raw method forms G , h , and q from $n+m$ rows, while the sufficient-statistics method forms contributions from only m rows because the old n rows have already been summarized. Dividing the two statistic-formation costs gives the stated ratio. $\square$

Including the solve term, the end-to-end ratio is

$$\frac{(n+m)p^2 + p^3}{mp^2 + p^3}.$$

When p^3 is small relative to mp^2 , this approaches 1001 for the given numbers. When p is so large that p^3 dominates both methods, the end-to-end ratio approaches 1 because both methods spend most of their time in the same solve.

2.2 Coefficients requested after every appended row

If coefficients are requested after each of the m appended rows, a raw refit after append j scans $n+j$ rows:

$$C_{raw,each} = \sum_{j=1}^m \Theta((n+j)p^2 + p^3) = \Theta(mnp^2 + m^2p^2 + mp^3).$$

The sufficient-statistics approach updates one row and solves at each step:

$$C_{suff,each} = \Theta(mp^2 + mp^3).$$

Theorem 2 (Append speedup for solve-after-each-append). *For fixed p , or whenever repeated row scans dominate dense solves, the sufficient-statistics method improves the append stream from $\Theta(mn)$ row work to $\Theta(m)$ row work, a $\Theta(n)$ statistics-maintenance speedup. With $n = 1,000,000$, the leading row-scan factor is one million.*

Proof. The raw method revisits approximately n old rows for each append, so its leading row-statistics term is mnp^2 . The incremental method touches only the new row for each append, so its leading row-statistics term is mp^2 . Dividing gives n . $\square$

3 Rolling window case: window size 1000

Let the rolling window size be

$$w = 1000.$$

For each slide, one row leaves and one row enters.

A raw rolling implementation that recomputes statistics for each window from the w active rows costs

$$C_{raw,roll}(T, w, p) = \Theta(T(wp^2 + p^3)).$$

A sufficient-statistics rolling implementation subtracts the departing row contribution and adds the arriving row contribution:

$$G^+ = G - x_{old}x_{old}^T + x_{new}x_{new}^T.$$

The matching h and q updates cost no more than the two row outer products, so the rolling sufficient-statistics cost is

$$C_{suff,roll}(T, p) = \Theta(T(p^2 + p^3)).$$

Theorem 3 (Rolling speedup). *Ignoring the common solve term, the rolling sufficient-statistics method reduces statistic maintenance from $\Theta(Twp^2)$ to $\Theta(Tp^2)$. The statistics-maintenance speedup is therefore*

$$\Theta(w).$$

For $w = 1000$, this is a 1000-fold asymptotic reduction in row contribution work.

Proof. The raw method processes all w rows in each of the T windows. The rolling sufficient-statistics method processes exactly two row contributions per slide: one subtraction and one addition. Constants do not affect Θ notation, so the ratio is $\Theta(w)$. $\square$

Including dense solves, the end-to-end cost ratio is approximately

$$\frac{wp^2 + p^3}{p^2 + p^3}.$$

For fixed p , this is $\Theta(w)$. If p is much larger than the window size, the dense solve can dominate and reduce the observable wall-clock speedup even though the row-statistics work still drops by the factor w .

4 Weighted, ridge, and forgetting costs

The sufficient-statistics formulas preserve the same asymptotic update costs for common OLS variants:

Variant	Additional work	Update cost
Weighted OLS	Multiply the row contribution by w_i .	$\Theta(p^2)$
Ridge	Add fixed penalty before solving.	Solve: $\Theta(p^3)$
Forgetting	Scale G by α , scale h and q , then add the new row.	$\Theta(p^2)$

The current dense implementation materializes and adds a $p \times p$ penalty matrix, so the ridge pre-solve setup is $\Theta(p^2)$ even when the penalty is diagonal. This term is lower order than the dense $\Theta(p^3)$ solve, but it is still the implementation's actual setup cost.

Exponential forgetting does not need to rescan historical rows. At time t , all historical decay is represented by scaling the previous statistics:

$$G_t = \alpha G_{t-1} + x_t x_t^T, \quad h_t = \alpha h_{t-1} + x_t y_t, \quad q_t = \alpha q_{t-1} + y_t^2.$$

Scaling G is $\Theta(p^2)$, matching the row outer-product order.

5 Memory complexity

A batch fit from raw dense data stores X and y with memory

$$\Theta(np + n).$$

An incremental sufficient-statistics fit stores

$$G, h, q, n,$$

with memory

$$\Theta(p^2 + p + 1) = \Theta(p^2).$$

Rolling windows need the summary plus enough information to know the departing row. Storing the active dense rows costs

$$\Theta(wp + w),$$

so rolling memory is

$$\Theta(p^2 + wp + w).$$

This is still independent of the total stream length.

6 Maintaining a factorization: rank-1 inverse updates

The analysis above maintains the sufficient statistics incrementally but still recovers coefficients with a fresh dense solve, which is $\Theta(p^3)$ on every request. The remaining $\Theta(p^3)$ term can be removed for the append and rolling cases by maintaining a factorization of the regularized Gram matrix in addition to the statistics. Let

$$A = G + R, \quad M = A^{-1},$$

where R is the fixed ridge penalty matrix. Because R does not change, every row contribution moves A by a single rank-1 term: an appended or departing weighted row (z, w) gives

$$A' = A + c z z^T, \quad c = \pm w$$

($+w$ to add, $-w$ to drop). The Sherman–Morrison identity updates the inverse directly:

$$M' = M - \frac{c}{1 + c z^T M z} (M z)(M z)^T.$$

Forming $M z$ is $\Theta(p^2)$, and the rank-1 correction is another $\Theta(p^2)$ outer product, so each update is $\Theta(p^2)$. Coefficients are then a matrix–vector product

$$\beta = M h,$$

which is also $\Theta(p^2)$.

Theorem 4 (Per-step cost with maintained inverse). *With an incrementally maintained inverse, appending a row and reading coefficients costs $\Theta(p^2)$ per step, and a rolling slide (one downdate plus one update) followed by a read costs $\Theta(p^2)$ per step. This removes the $\Theta(p^3)$ per-read solve from the solve-after-each append stream and from the rolling stream.*

Proof. Each step performs a constant number of matrix–vector products and rank-1 outer products on $p \times p$ data, each $\Theta(p^2)$, and no dense $p \times p$ solve or factorization from scratch. $\square$

Three practical points bound the result. First, M exists only once A is invertible, so an implementation either seeds it from a ridge prior or builds it lazily once enough rows accumulate; rows added before that point update only G and are folded in by the first build. Second, repeated Sherman–Morrison updates accumulate floating-point error, and downdates are the unstable direction, so a periodic exact refactorization every K updates caps the drift at an amortized $\Theta(p^3/K)$ per step while keeping the per-step cost $\Theta(p^2)$ for the common case. Third, a downdate whose denominator $1 + c z^T M z$ collapses signals that removing the row makes A singular; the maintained inverse is then discarded and rebuilt from the exact G , so the singular case still fails loudly rather than propagating a blown-up inverse.

This trick is the classical recursive-least-squares update and is prototyped in `sufficient_regression.rank_one` (`RankOneIncrementalOLS`, `RankOneRollingOLS`). Exponential forgetting is not covered there: its ridge term is not decayed alongside G , so the regularized system does not reduce to a single clean rank-1 recursion.

7 Interpretation

For the million-plus-1000 append case, the sufficient-statistics advantage comes from not rereading the one million previously summarized rows. For the 1000-window rolling case, the advantage comes from replacing each full-window rebuild with one subtract and one add.

The headline asymptotic speedups are exact for statistic maintenance: 1001 for a final million-plus-1000 append rebuild, $\Theta(n)$ for solve-after-each append streams, and $\Theta(1000)$ for a rolling window of size 1000. With a cold dense solve on every request, end-to-end wall-clock speedups are bounded by the unchanged dense solve cost; the rank-1 inverse-maintenance prototype of Section 6 lifts that bound for the append and rolling cases by replacing the per-read $\Theta(p^3)$ solve with a $\Theta(p^2)$ update and matrix–vector product.